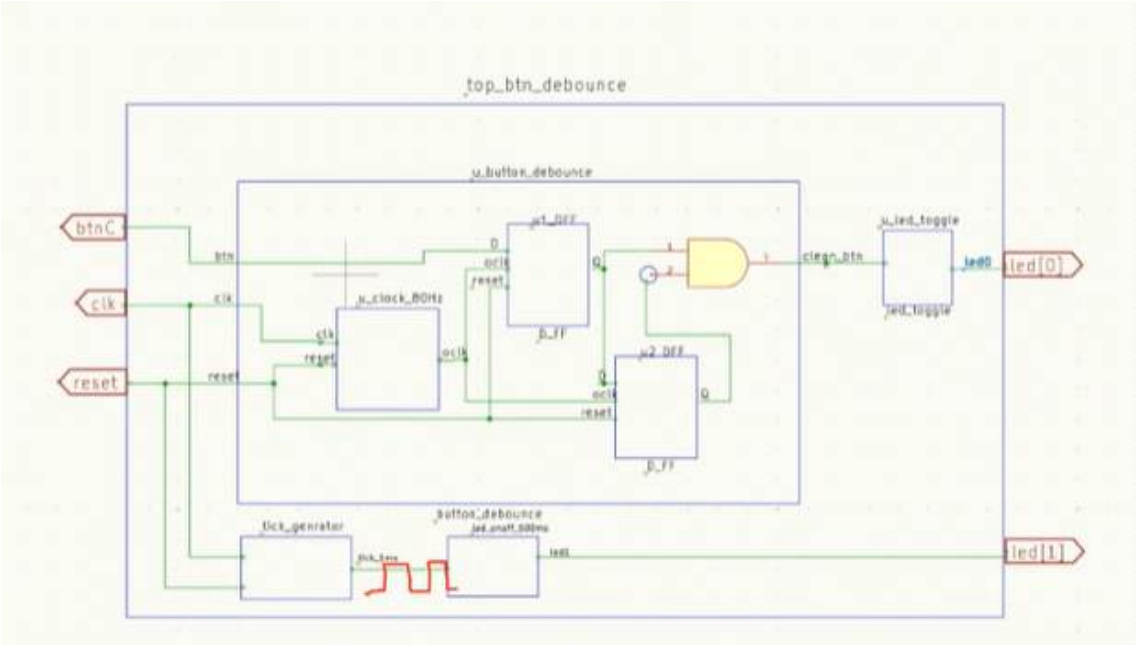
10/15 my_tick_ledonoff 박수민



[top_btn_debounce.v]

```
`timescale 1ns / 1ps

module top_btn_debounce(
    input clk,
    input reset,
    input btn,
    output [1:0] led
    );

    wire w_clean_btn;
     wire w_tick_1ms;

    button_debounce u_button_debounce(
        .clk(clk),
        .reset(reset),
        .btnC(btn),
        .clean_btn(w_clean_btn)
    );

    led_toggle u_led_toggle(
```

```
        .clean_btn(w_clean_btn),
        .led(led[0])
    );

    tick_generator u_tick_generator(
        .clk(clk),
        .reset(reset),
        .tick_1ms(w_tick_1ms)
    );

    led_onoff_500ms u_led_onoff_500ms(
        .clk(clk),
        .reset(reset),
        .tick_1ms(w_tick_1ms),
        .led(led[1])
    );

endmodule
```

[led_tick_ledonoff.v]

```
`timescale 1ns / 1ps


module led_onoff_500ms(
    input clk,
    input reset,
    input tick_1ms,
    output reg led
    );

    reg [8:0] r_count = 0;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_count <= 0;
            led <= 0;
        end
        else if (tick_1ms) begin
            // 500ms 마다 led 점멸
            if(r_count == 499) begin
```

```
                r_count <= 0;
                led <= ~led;
            end
            else begin
                r_count <= r_count +1;
            end
        end

    end

endmodule
```

[tick_generator.v]

```
`timescale 1ns / 1ps

module tick_generator(
    input clk,
    input reset,
    output reg tick_1ms
    );

    reg [16:0] r_counter = 0;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_counter <= 0;
            tick_1ms <= 0;
        end
        else begin
            if (r_counter == 99999) begin
                r_counter <= 0;
                tick_1ms <= 1;
            end
            else begin
                r_counter <= r_counter + 1;
                tick_1ms <= 0;
            end
        end
    end

endmodule
```